

Kasbana — Full Project Plan

The complete build, end to end: four stages from backend to pilot launch, broken into phases, with the work split across two developers so it merges without conflict.

KASBANA • كسبانه — LOYALTY & WALLET-PASS PLATFORM

Backend • Dashboard • Admin • Launch

Stages

4

Team

2 devs — You + Joe

v1 scope

Stamp cards

Pricing

Trial → 3 tiers

Overview & locked decisions

The project is four stages; each stage is a set of phases gated by exit criteria, not dates. Stages 1 and 2 are split across two developers and built in parallel; Stages 3 and 4 are not split. Everything sits on the frozen backend contract, so parallel work always merges clean.

LOCKED DECISIONS (DRIVE THE WHOLE PLAN)

v1 card types: stamp cards only ("buy N, get 1 free"). Points and tiers/membership are deferred to a post-launch growth phase. **Pricing:** a 14-day free trial, then Starter / Growth / Chain — no free-forever tier.

Admin model: a full-access admin account (you can see and edit everything to support any merchant directly — no "log in as merchant" needed), plus refunds & billing control, a fraud/abuse review queue, and a feature-flag / plan editor. **Team:** two developers (You + Joe); Stages 1 & 2 are split, Stages 3 & 4 are not.

The four stages

- 1
Backend 2 devs · split Foundation, wallets, loyalty engine, dashboard API, billing. Merge = full backend.
- 2
Client Dashboard 2 devs · split Card designer, customers & analytics, messaging, billing/plan UI.
- 3
Admin Panel 1 dev · not split Your control center — full access, billing, fraud, feature flags.
- 4
Finalize, integration & testing not split Wire everything together, test end to end, launch the pilot.

Team & parallelization model

Two developers with overlapping skills. The one fixed rule: **You own server & infrastructure**. Beyond that, work is split by clean module ownership so the two of you rarely touch the same files.

You

Platform · Wallets · Infra · Billing

Backend foundation, both wallet integrations, billing/subscriptions, messaging, and all infrastructure. On the dashboard, the create-and-engage side.

Joe

Loyalty engine · Product API · Clients

The loyalty engine + anti-fraud, the dashboard/analytics API, and the measure-and-manage side of the client dashboard.

WHY IT MERGES WITHOUT CONFLICT (RECAP)

It rests on the four mechanisms from the backend contract: a **frozen API + variable contract** (same names everywhere), **module ownership** (disjoint folders), **interface stubs** (code against a seam before it's implemented), and a **per-phase integration checkpoint**. Stage 2 reuses the same idea on the frontend: a shared design system + the frozen API contract, with each dev owning separate feature folders.

Stage	Split?	How it's divided
1 · Backend	2 devs	You = enrollment + wallets + billing · Joe = loyalty engine + dashboard API. Merge = full backend.
2 · Dashboard	2 devs	Dev A = designer + messaging · Dev B = customers + analytics + billing UI. Shared frontend foundation.
3 · Admin	1 dev	Built by whoever frees up first; reuses the backend with elevated access.
4 · Finalize	together	Both devs converge for integration, testing, and launch.

Backend

Detailed fully in the Backend Plan & Variable Contract. Below is the phase split. Because v1 is stamp-only, the loyalty engine stays simple — no points math.

You Platform/Wallets/Billing **Joe** Loyalty/API ♦ **integration**

Phase	You	Joe	Integration ♦
1.0 Foundation	Shared — build first, then freeze. Scaffold, core models, enums, constants, JWT/RBAC, tenancy, the loyalty ledger, wallet interface stubs, and the frozen <code>openapi.yaml</code> .		♦ Tag core-v1 ; both branch from here.
1.1 Enrollment + Wallets	Enrollment tokens + endpoints; Google + Apple provisioning, pkpass signing, the Apple web service, APNs. Implements the wallet façade.	—	♦ Enroll → card in both wallets, live updates.
1.2 Loyalty engine	—	Stamp / redeem endpoints (stamp-only), anti-fraud (cooldown, velocity, staff/location binding). Calls the ledger + wallet façade.	♦ Stamp updates pass live; fraud guards hold.
1.3 Dashboard API	—	Card CRUD, customers, analytics aggregations over the ledger — the API the client dashboard consumes.	♦ Dashboard endpoints live against contract.
1.4 Billing + Messaging	Paymob/Fawry subscriptions + the 14-day trial logic; the entitlements engine that enforces plan limits; WhatsApp senders.	—	♦ Trial→paid works; plan limits enforced server-side.

STAGE 1 EXIT

Both branches merged: a merchant can be created, a stamp card enrolled to both wallets, stamped live, billed through a trial into a paid plan, with plan limits enforced. The backend is whole.

Client Dashboard

The merchant's web app. Built by two devs on a shared frontend foundation, against the frozen API contract. Feature scope is benchmarked to the leaders (Loopy Loyalty, LoyaltyPass) plus our differentiators.

Dev A Design & engage Dev B Measure & manage ♦ integration

Phase	Dev A	Dev B	Integration ♦
2.0 Foundation	Shared. React app shell, design system + tokens, Arabic/RTL, auth, the typed API client generated from the contract, and a mock server.		♦ Shell deploys; both build on it.
2.1 Card designer	WYSIWYG card designer — logo, colours, stamp count, reward — with live Apple & Google Wallet preview; brand asset management.	—	♦ Design a card → provisions via API.
2.2 Enroll & engage	Enrollment QR / link management; messaging & campaigns UI (push + WhatsApp + automations: expiry, birthday, win-back).	—	♦ Send a campaign; generate a join QR.
2.3 Customers & analytics	—	Customer list & profiles (history, visits), segmentation; analytics dashboards (joins, stamps, redemptions, retention, Apple/Google split).	♦ Live data renders from the dashboard API.
2.4 Billing & settings	—	Subscription/billing UI, plan-gating UX (lock + upgrade prompts), team/staff, locations, account settings.	♦ Upgrade flow + gating reflect entitlements.

STAGE 2 EXIT

A merchant can self-serve end to end: design a card, enroll customers, run campaigns, read analytics, manage team & locations, and manage their subscription — all gated correctly by plan.

Dashboard feature catalog

Benchmarked to the wallet-loyalty leaders. **std** = competitors have it · **+ours** = our addition / Egypt edge.

Card design

std WYSIWYG designer, dual-wallet live preview, configurable stamp count (to 30), single + multi-reward milestone cards. **+ours** Arabic-first / RTL designer.

Enrollment

std QR + link join, no customer app, custom fields incl. birthday, consent. **+ours** WhatsApp enrollment link + PDPL consent.

Stamping (staff)

std scan to add/remove stamps, multi-reward redeem in one scan, status/authenticity check, GPS-logged transactions, per-customer history. **+ours** anti-fraud on every tier.

Customers

std profiles (history, visits, messages), segmentation. **+ours** CSV export from mid-tier (not locked to top tier).

Messaging

std wallet lock-screen push, expiry/reward/birthday/win-back automations, geofence/beacon alerts. **+ours** WhatsApp campaigns & notifications.

Analytics

std joins, stamps, redemptions, real-time audit feed. **+ours** retention curves, at-risk alerts, Apple-vs-Google split.

Multi-location & team

std locations, staff sub-users, multiple campaigns at once.

Integrations

std Zapier + REST API (Square, Shopify, Mailchimp) — later phase. **+ours** mobile-friendly dashboard.

DEFERRED TO POST-LAUNCH (NOT IN V1)

Points programs, tiers/memberships, cashback, POS auto-stamping, Zapier/API, NFC Smart Tap. The catalog above is the v1 stamp-card scope; these are the growth backlog.

Subscriptions & feature gating

How limited access per plan actually works — the part to get right early, because it touches all three of backend, dashboard, and admin.

The mechanism (five moving parts)

1. **An entitlements map** — one central table: `plan → {limits + features}`. The single source of truth for what each plan can do.
2. **Backend enforces it (the real gate)**. Every action checks the merchant's entitlements first — a 2nd location on Starter is rejected with a `PLAN_LIMIT` error. Can't be bypassed.
3. **Frontend reflects it (UX only)**. The dashboard locks/greys features and shows "Upgrade to unlock". Presentation, never security.
4. **Billing syncs the plan**. Paymob/Fawry webhooks flip the merchant's plan on subscribe / upgrade / downgrade / cancel; the trial timer drives the trial→paid transition.
5. **Admin can edit entitlements** via the feature-flag / plan editor — no code deploy to change a limit or grant an exception.

Plans (14-day trial, then 3 tiers)

The trial is full Growth-level access for 14 days so merchants feel the whole product; at expiry they pick a paid plan or the account locks (data retained). Figures are anchors to validate with real merchants.

Capability	Trial (14d)	Starter	Growth	Chain
Card designs	all	1	up to 5	unlimited
Locations	all	1	up to 5	unlimited
Staff users	all	3	15	unlimited
Active customers	full	~500	~5,000	unlimited
Wallet push	✓	✓	✓	✓
WhatsApp	✓	basic	higher	highest
Analytics	full	basic	full	full
CSV export	✓	—	✓	✓
Campaigns / automations	✓	limited	✓	✓
API / Zapier	—	—	—	✓
Support	standard	standard	priority	SLA

BUILD NOTE

The backend already carries the hook (`Merchant.plan` + trial timestamp). Build the entitlements engine in Phase 1.4 and have the dashboard read the same entitlements object so the UI and the server never disagree.

Admin Panel

Your control center — a single full-access account that lets you see and edit everything to support any merchant directly. Not split; built by whoever frees up after Stages 1–2.

Phase	Scope	Detail
3.0	Admin foundation req	Admin auth (separate from merchant auth), the full-access permission model, admin app shell.
3.1	Merchant management req	View / create / suspend merchants; full edit access to any merchant's data & settings to fix problems directly (your chosen support model — no impersonation needed).
3.2	Billing & revenue req + optv	Subscription & revenue oversight; refunds & manual billing control (chosen). Trial conversions, MRR, churn.
3.3	Feature-flag / plan editor optv	Edit the entitlements map, adjust limits, grant per-merchant exceptions — no deploy.
3.4	Fraud / abuse review optv	Queue of flagged stamping activity (velocity, duplicates, suspicious GPS); review, block, or clear.
3.5	Platform health req	Platform-wide analytics, system + audit logs, wallet-credential health (Apple/Google cert & key expiry alerts).

REQUIRED VS OPTIONAL

Required: 3.0, 3.1, 3.5 — you can't run the platform without merchant management, support access, and health visibility. **Chosen optional:** refunds & billing control (3.2), feature-flag editor (3.3), fraud queue (3.4). Dropped: "log in as merchant" — replaced by full admin access per your decision.

STAGE 3 EXIT

You can manage every merchant, resolve billing and support issues, tune plans, review fraud, and watch platform + credential health — all from one panel.

Finalize, integration & testing

Both developers converge. Everything built across Stages 1–3 gets wired together, tested end to end, and taken to a pilot launch.

4.0

End-to-end integration

Wire all stages together on staging and prove the full path runs.

- Connect dashboard + admin to the live backend; confirm the frozen contract held across all teams.
- Run the complete journey: merchant signs up → trial → designs card → customer enrolls → staff stamps → reward redeemed → merchant billed → admin oversees.

EXIT

The whole journey runs on staging without manual patching.

4.1

QA & test passes

Systematically test every critical path.

- Core loop on **both wallets** (Apple live update + Google auto-push); anti-fraud guards; plan-gating at every limit; billing webhooks & trial→paid→cancel; Arabic/RTL across all surfaces.
- Unit + contract + integration tests green; cross-browser + real-device checks (iPhone + Android).

EXIT

All critical paths pass; no open blocker bugs.

4.2

Security, PDPL, performance & observability

Make it production-grade.

- Security review (RBAC, tenant isolation, secrets, per-pass tokens); PDPL consent & retention verified; load/perf on stamping + pass-push throughput.
- Sentry, structured logs, metrics, uptime alerts, backups + restore drill.

EXIT

Monitoring, backups, and security all verified.

Go live with one real merchant.

- Promote to production; confirm Apple review + Google publishing access are cleared (or launch Google-only and add Apple on clearance).
- Onboard the pilot café; staff on-site day one; monitor logs & metrics; tight feedback loop.

EXIT

Real customers enroll, collect stamps live, and redeem — no manual intervention per transaction. Pilot is live.

Sequencing & critical path

Stages overlap deliberately — the only hard serial dependency is the backend foundation. Everything downstream builds against the frozen contract while the backend fills in behind it.

Rule	Why
Phase 1.0 first, frozen	The contract + core unblock every other stream. Nothing parallel starts until <code>core-v1</code> is tagged.
Stage 2 starts in parallel	The dashboard's foundation (2.0) builds against the frozen contract + mock the moment 1.0 is done — no need to wait for the backend to finish.
Backend before its consumers	Dashboard analytics (2.3) needs the dashboard API (1.3); billing UI (2.4) needs the billing backend (1.4). Sequence those pairs.
Admin after backend core	Stage 3 reuses the backend with elevated access; it can begin once 1.0–1.2 land, overlapping the tail of Stage 2.
Stage 4 is last & joint	Integration + testing need 1–3 substantially complete; both devs converge here.

CRITICAL PATH TO PILOT

1.0 → (1.1 + 1.2 in parallel) → 1.3/1.4 || 2.0 → 2.1/2.3 → integrate → Stage 4. Admin (Stage 3) and the remaining dashboard phases run alongside and fold in before 4.0. Apple/Google approval gates run in the background from day one so they never block the launch.

TOP RISKS (CARRIED FROM EARLIER DOCS)

The two approval gates (Apple review, Google publishing) · Apple web-service/APNs complexity · stamp fraud · scope creep before the core loop works · WhatsApp cost vs. plan allowances. Mitigations: start gates early, de-risk Apple on one device first, server-side anti-fraud, treat the Google-only stamp loop as the launch scope, meter WhatsApp per tier.

Companion documents: *Product Blueprint · Technical Development Plan · Team Split & Parallel Execution · Backend Plan & Variable Contract*. This master plan supersedes the earlier working name — the product is now *Kasbana* (كسبانه).